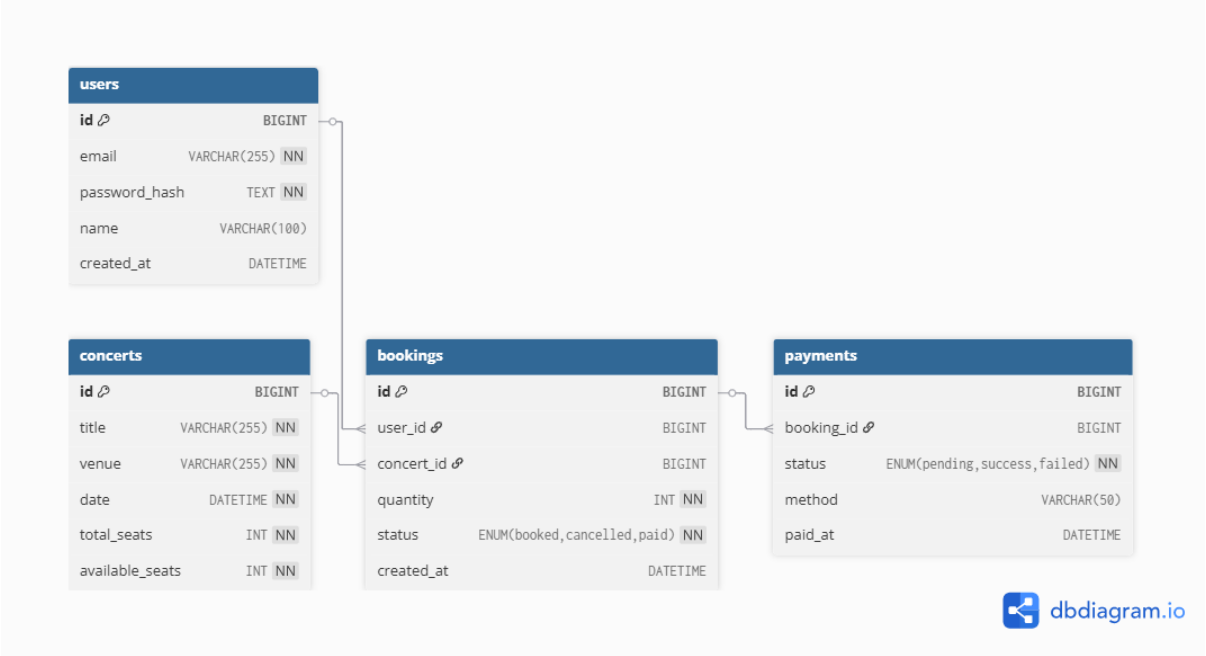
Soal Tambahan

1. SQL & Index Optimization

Query SQL dan penjelasan optimasi index.

- Membuat struktur database relasional berdasarkan project **Concert Ticket Booking System** diatas.



- Menjelaskan **strategi optimasi query** dengan cara menambahkan **index** yang tepat untuk meningkatkan performa pada dataset yang sangat besar.

  Index Optimization

  - Query umum: ambil semua pemesanan user → WHERE user_id = ?

  - Tambahkan index :

```
CREATE INDEX idx_bookings_user_id ON bookings(user_id);

CREATE INDEX idx_bookings_concert_id ON bookings(concert_id);

CREATE INDEX idx_payments_booking_id ON payments(booking_id);
```

2. High-Load Optimization

Cara mengurangi beban ke DB saat endpoint menerima 100.000 hit/detik.

Solusi jika 100.000 hit/detik

- Gunakan Redis cache untuk data yang tidak sering berubah (list konser).
- Rate limiting per IP dengan middleware.
- Load balancer (Nginx/HAProxy).
- Connection pooling (gunakan sql.DB dengan pengaturan max idle/open).
- Queue untuk heavy operation (Kafka, RabbitMQ, MQTT).

- Gunakan batching kalau memungkinkan.

3. API Security
Sebutkan minimal 5 langkah mengamankan endpoint POST /api/users.

- Rate Limiting (hindari brute-force).
- Input validation (hindari SQL injection/XSS).
- HTTPS only.
- JWT Auth & Session management.
- CSRF Token.
- Limit content-type ke application/json.
- Log dan audit trail.
- ReCaptcha (untuk public registration).

4. CI/CD Workflow
Jelaskan secara lengkap dan terstruktur alur Continuous Integration dan Continuous Deployment (CI/CD) yang terjadi mulai dari developer melakukan Git commit hingga aplikasi ter-deploy ke lingkungan production.

Contoh Workflow:

- Developer push ke main branch.
- GitHub Actions / GitLab CI: Jalankan unit test, Build image Docker tiap microservice, Push image ke Docker Hub/GitHub Container Registry.
- Jalankan Deployment: Jika lolos test, jalankan deploy via ssh, rsync, atau push ke cluster Kubernetes/EC2/VPS.
- Monitor hasil via log dan alert.

5. Code Review & Debugging (Golang)
Berikut potongan kode handler API menggunakan Gin:

```
r.GET("/user/:id", func(c *gin.Context) {
    id := c.Param("id")
    user, err := db.FindUserByID(id)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error":
"Internal server error"})
        return
    }
    if user == nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "User not
found"})
        return
    }
    c.JSON(http.StatusOK, user)
})
```

Instruksi:
- Identifikasi minimal 3 masalah/bug dari kode awal.

- Jelaskan kesalahan dan perbaikannya.
- Tunjukkan versi kode yang sudah diperbaiki.

Masalah:

- ID tidak divalidasi (raw string, bisa inject non-numeric).
- Error ambiguous: InternalServerError diberikan tanpa rincian/debug log.
- Tidak menangani context timeout atau DB context.

Versi Diperbaiki:

```
r.GET("/user/:id", func(c *gin.Context) {

    idStr := c.Param("id")

    id, err := strconv.Atoi(idStr)

    if err != nil {

        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid user
ID"})

        return

    }

    ctx, cancel := context.WithTimeout(c.Request.Context(),
3*time.Second)

    defer cancel()

    user, err := db.FindUserByID(ctx, id)

    if err != nil {

        log.Printf("DB error: %v", err)

        c.JSON(http.StatusInternalServerError, gin.H{"error":
"Database error"})

        return

    }

    if user == nil {

        c.JSON(http.StatusNotFound, gin.H{"error": "User not
found"})

        return

    }

    c.JSON(http.StatusOK, user)
```

```
})
```